

Phase 7 - limitations

- [PHASE 7 — LIMITATIONS](#)

- [1. Everything in this phase is UNCALIBRATED, unconditionally](#)
- [2. Distribution bounds are documented judgement calls](#)
- [3. Only one correlation is modeled, and it too is a judgement call](#)
- [4. The decision-stability classifier is intentionally minimal and financial-only](#)
- [5. `holding\_period\_years` is not randomized by default](#)
- [6. Purchase price gets only a narrow shock](#)
- [7. Same OBSERVED-data coverage gap as Phase 6](#)
- [8. No live PostgreSQL was available in this sandbox](#)
- [9. Simulation runtime at full-portfolio scale is not measured](#)
- [10. IRR can be undefined for individual simulated draws](#)
- [11. Post-delivery audit fix: inherited the Phase 6 ROI bug \(now fixed\)](#)

PHASE 7 — LIMITATIONS

Consolidated from the phase report, per Master Prompt Section 16.

1. Everything in this phase is UNCALIBRATED, unconditionally

No repeated-period panel exists in this dataset for any single property (no time series of one unit's vacancy, no multiple resales of the same unit), so there is no way to fit a distribution's parameters to real historical outcomes. Every `MonteCarloResult` this phase produces carries `calibration_status = "UNCALIBRATED"` with no exception, per §7.7. The probabilities reported (e.g. "38% mean probability of negative NPV across the demo batch") describe what *this simulation, with these documented assumptions* produces — they are not validated real-world likelihoods, and should not be quoted as such downstream (including by Phase 9 or any LLM explanation layer).

2. Distribution bounds are documented judgement calls

`monte_carlo.DEFAULT_SPECS` (e.g., `vacancy_rate: -50%/+150%` multiplicative shock; `appreciation_rate: ±6` percentage points additive) are stated with inline reasoning for each range but are not fitted to anything. A different, equally defensible analyst could reasonably choose different bounds. This is disclosed explicitly rather than presented with false precision (e.g. as if derived from a fitted standard deviation).

3. Only one correlation is modeled, and it too is a judgement call

Vacancy rate and operating expenses are drawn with $\rho=0.4$ via a Gaussian copula — a plausible business relationship (weak local markets tend to show elevated vacancy and elevated turnover/marketing/maintenance cost together), not fitted to this dataset (no panel data exists to fit it from). Purchase price, appreciation rate, and interest rate are sampled independently of everything else and of each other; no correlation is claimed for them because none could be justified from available data. A real deployment with genuine historical panel data should replace this with a fitted correlation structure and drop the UNCALIBRATED status only once that fitting is actually done and validated.

4. The decision-stability classifier is intentionally minimal and financial-only

`decision_stability.classify_financial_outcome()` is a three-branch rule (INVEST if $IRR \geq \text{target}$ and $NPV > 0$; AVOID if both negative; HOLD otherwise) built solely to answer §7.8's literal question about simulated decision stability *on the financial dimension*. It is explicitly **not** Phase 9's authoritative

`DecisionPolicy` (\$9.4), which will combine market, prediction, financial, risk, and geo scores with configurable weights. Using this phase's label as a final investment decision would itself be a governance violation — the module docstring and every result's `scope_note` field say so directly.

5. `holding_period_years` is not randomized by default

Holding period is treated as a decision the investor makes (how long they choose to hold), not a market uncertainty, so it is not perturbed in the default `MonteCarloConfig`. A caller who wants to treat holding period as uncertain (e.g. for a fund with an uncertain exit window) can add a spec for it; the plumbing for adding new sampled variables is generic and doesn't require touching `monte_carlo.py`'s core loop.

6. Purchase price gets only a narrow shock

The purchase-price Triangular spec (-5%/+3%) reflects negotiation-range uncertainty around a price the buyer already knows (from `asking_price` or an explicit override), not genuine market-price volatility — because market-price volatility is already captured separately, and more directly, through the `appreciation_rate` variable driving exit value. Widening this further would double-count the same underlying uncertainty in two places.

7. Same OBSERVED-data coverage gap as Phase 6

The demo batch (n=25) only includes properties meeting Phase 6's eligibility bar (both expense and rental history present, reliable locality appreciation) — 44.5% of the 8,000-property portfolio. Properties outside that bar will return `INSUFFICIENT DATA` from Monte Carlo, exactly as they do from the Phase 6 engine, unless a caller supplies the missing inputs explicitly.

8. No live PostgreSQL was available in this sandbox

Same situation as Phases 3, 5, and 6. All orchestrators default to `--source db` (the production path) but were run here with `--source csv`, reading the identical rows Phase 1 loaded. Recommend re-running with `--source db` before Phase 9 integration to confirm the DB path end-to-end.

9. Simulation runtime at full-portfolio scale is not measured

The n=25 demo batch (50,000 total financial-engine calls across Monte Carlo draws) completed in a few seconds in this sandbox. No claim is made about runtime or memory behavior at the full 8,000-property scale; per \$15, that should be measured before any optimization decision, not assumed.

10. IRR can be undefined for individual simulated draws

Some simulated cash-flow series (e.g., a draw with very high sampled expenses and low sampled appreciation) can produce a cash-flow schedule with no sign change, making IRR mathematically undefined for that specific draw (same reasoning as Phase 6's `irr()` function). These draws' IRR is `None` and they are excluded from the IRR distribution's mean/std/percentiles (`n_irr_undefined_in_valid_draws` reports how many, per property) rather than being dropped silently or coerced to a number.

11. Post-delivery audit fix: inherited the Phase 6 ROI bug (now fixed)

`src/finance/engine.py` was copied into this phase from Phase 6 *before* Phase 6's own post-delivery audit found and fixed a bug in `roi_total_holding_period` (it was computing a gross cash multiple instead of a net return, overstating ROI by exactly 100 percentage points in every case). Because this phase's Monte Carlo, risk-metrics, and extended-scenario modules all pull `roi_total_holding_period` straight from `engine.run_financial_engine()`'s output (per Section 7.2's reuse requirement), the bug silently propagated into every simulated ROI draw and into `docs/risk_results_csv/05_extended_scenario_comparison.csv`. **Fixed** by re-copying Phase 6's corrected

`engine.py` (re-verified byte-identical via `diff`), with one new regression test (`test_roi_distribution_is_net_return_not_gross_multiple`) added to `test_phase7.py` - suite is now 21/21 passing. No other Phase 7 metric (IRR, NPV, VaR, CVaR, probabilities, risk drivers, decision stability) was affected, since none of them read `roi_total_holding_period`.